Urban Data Integration Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

September 13, 2026

1 Introduction

This project aims to build a reusable urban data integration platform using Apache Spark and Delta Lake. The platform processes four heterogeneous datasets from four datasets differ in schema, naming conventions, temporal representation, and update patterns into an analytical repository.

2 Data Catalog

Before implementing the platform, we examined the structure and meaning of each dataset. Table 1 summarizes their key characteristics, including identifiers, join attributes, temporal and categorical attributes, and expected growth patterns.

Table 1: Data Catalog for the four source datasets

Dataset	Entity	Candidate Key	Join Attributes	Temporal Attributes	Categorical Attributes	Growth
Taxi Trips	Completed taxi trip	Composite `trip_id` from vendor, time, location, distance, and amount	`PULocationID`, `DOLocationID`, pickup time	Pickup and dropoff timestamps	Vendor, rate code, location, payment type	New trips appended continuously
Weather	Hourly weather observation	Year, month, day, hour	Hourly timestamp	Year, month, day, hour	Weather-source fields	New observations added hourly
Air Quality	PM2.5 measurement at a site	Station, parameter, method, and timestamp	Time and location	Local/GMT timestamps	Station, parameter, method, location	New measurements added over time
Taxi Zone Lookup	TLC taxi zone	`LocationID`	`LocationID`	None	Borough, zone, service zone	Mostly static

Taxi trips are the main fact dataset, each record representing a completed yellow-taxi trip. Since the source has no native trip identifier, a composite key is used to construct a stable `trip_id`. Taxi trips are linked to Taxi Zone Lookup by location and to Weather and Air Quality by time.

Weather and Air Quality provide time-dependent observations that can be integrated with taxi trips at an hourly level. Both datasets grow continuously as new observations are collected.

Taxi Zone Lookup is a small, mostly static reference dataset that provides geographical information for taxi pickup and dropoff locations.

3 Storage Architecture

The storage architecture separates data according to its processing state:

```
data/
|-- raw/
|-- processed/
|-- failed/
‘-- metadata/
```

The `raw` directory stores immutable CSV and Parquet source files. Valid standardized records are stored as Delta tables in `processed`, while rejected records and their validation errors are stored in `failed`. The `metadata` directory contains ingestion statistics, schema information, and execution status.

Taxi Zone Lookup is the only dataset treated as a lookup table because it provides relatively static zone information referenced by Taxi Trips through location IDs. Weather and Air Quality provide contextual information, but they are time-series observation datasets rather than lookup tables because new records are continuously added.

Taxi Trips is partitioned by `pickup_year` and `pickup_month`, while Air Quality is partitioned by `observation_year` and `observation_month`. These large datasets grow over time and are commonly filtered by date, so monthly partitioning enables partition pruning without creating excessively small partitions. Weather is not partitioned because it is comparatively small, and Taxi Zone Lookup is not partitioned because it contains only a small number of reference records.

Partitioning becomes harmful when a dataset is small, the partition key has very high cardinality, partitions are unevenly distributed, or queries do not filter on the partition columns. Overly fine-grained daily or hourly partitions may also create many small files and increase metadata-management costs.

If the total data volume increased by 20 times, the platform would retain time-based partitioning for the large datasets, introduce regular file compaction and target file-size management, and monitor partition sizes and data skew. More granular partitioning would only be introduced if measured query patterns demonstrated a clear benefit.

4 Common Data Model

The common data model defines four rules applied before data is written to the processed layer.

Table 2: Common data model rules

Area	Common Rule
Timestamp format	Timestamps use Spark `TimestampType` and are displayed as `yyyy-MM-dd HH:mm:ss`. The session time zone is `America/New_York`. Air Quality retains both local and GMT timestamps.
Naming convention	All processed columns use lowercase `snake_case`. Unclear abbreviations are expanded, for example, `PULocationID` → `pickup_location_id`.
Missing values	Optional missing values remain `null`. Missing required fields, invalid timestamps, invalid numerical values, and duplicate keys are written to `failed`.
Common data types	Dates use `DateType`; numeric IDs use integer or long; measurements, distances, durations, and monetary values use double. Air Quality codes remain strings to preserve leading zeroes.

Dataset-specific transformations are summarized below. Taxi Trips generates `trip_id`, trip duration, and pickup date fields. Weather combines year, month, day, and hour into `observation_timestamp`. Air Quality parses local and GMT timestamps, generates `station_id` and `observation_id`, and removes the original time components. Taxi Zone Lookup converts `location_id` to integer and trims its text fields. Detailed grouped transformations are documented in `reports/common_data_model.md`.

5 Ingestion Framework

Figure 1 illustrates the generic ingestion framework and data flow. Blue stages represent reusable processing logic, while orange stages contain dataset-specific rules.

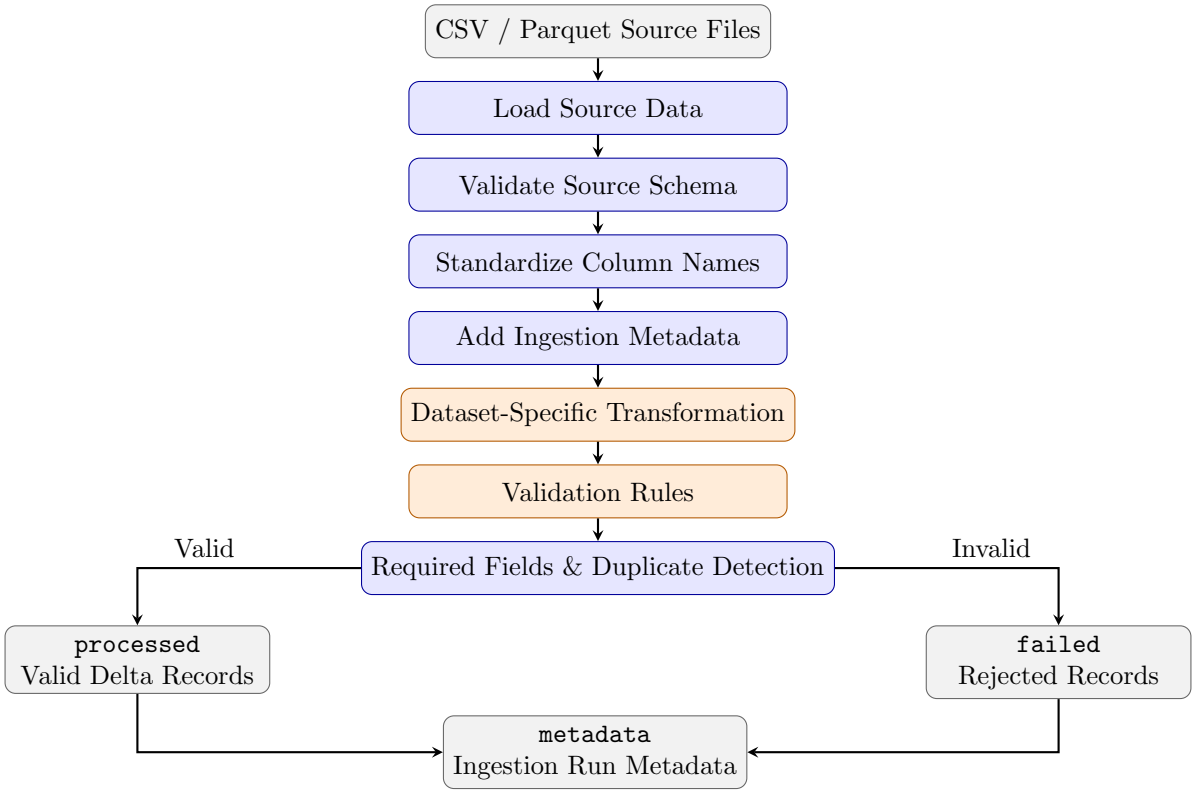


Figure 1: Generic ingestion framework and data flow. Blue stages represent reusable processing logic, while orange stages contain dataset-specific rules.

As shown in Figure 1, the reusable framework loads CSV or Parquet files, validates source schemas, standardizes column names, adds ingestion metadata, separates valid and failed records, and writes Delta outputs. These operations are implemented once and reused for all datasets, reducing code duplication. Transformation rules that depend on dataset semantics remain dataset-specific. These include timestamp construction, data-type conversions, identifier generation, derived fields, and numerical validity ranges.

Valid records are stored in `processed`, while invalid and duplicate records are stored in `failed` with `_validation_errors`. Each execution also appends a record to the metadata Delta table containing the schema version, input, processed, rejected, and duplicate counts, execution time, source and output schemas, and final status.

If 20 new datasets were added, the core ingestion workflow would remain unchanged. Each new dataset would require a configuration entry, expected schema, transformation function, and validation rules; simple datasets could reuse existing functions.

6 Integration Pipeline

Taxi Trips is the central fact table of the integration pipeline. Of 9,554,778 source records, 9,417,689 passed validation and 137,089 (1.43%) were stored in the failed layer. Since the source has no native identifier, a reproducible `trip_id` is generated by hashing selected trip attributes.

As shown in Figure 2, valid trips are enriched through left joins so that missing contextual data does not remove trips. Pickup and dropoff location IDs are joined to Taxi Zone Lookup. Weather is matched by pickup hour, while PM2.5 observations are aggregated by borough and hour before

being joined to the pickup borough and time.

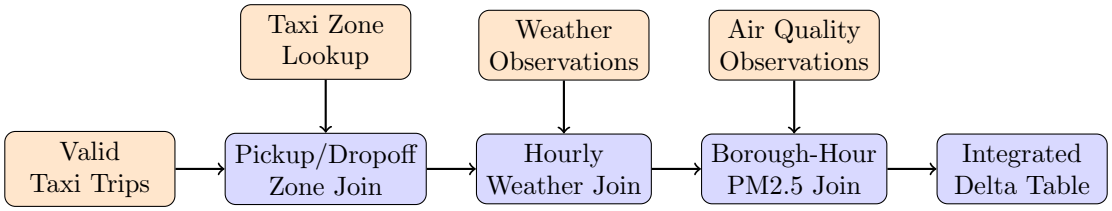


Figure 2: Taxi-trip integration pipeline. Orange nodes are source datasets, while blue nodes represent integration stages and output.

The output contains all 9,417,689 valid trips. Pickup-zone, dropoff-zone, and weather match rates reached 100%, while air-quality coverage was only 9.95% because an exact borough-hour observation was required. Missing context remains `NULL` and must not be interpreted as zero.

The pipeline is limited by coarse borough-level air-quality aggregation, pickup-time-only context, and full-table batch processing. Taxi ingestion took approximately 968.5 seconds because validation, duplicate detection, record counting, and Delta writes require several complete Spark jobs.

7 Conclusions

This project demonstrates a reusable urban data platform built with Apache Spark and Delta Lake. Four heterogeneous datasets were standardized through a common data model and processed by a configuration-driven ingestion framework. The architecture separates raw, processed, rejected, and metadata layers, preserves invalid records for auditing, and records execution and schema information for every ingestion run.

The integration pipeline successfully preserved all 9,417,689 valid taxi trips while enriching them with zone, weather, and air-quality context. Zone and weather integration achieved complete temporal and reference coverage for the current data, whereas the 9.95% air-quality match rate revealed the limitations of sparse monitoring data and strict hourly matching.

Future work should introduce incremental ingestion, Delta `MERGE`-based updates, file compaction, and improved monitoring. Air-quality coverage could be evaluated using nearest-time matching or daily borough aggregates, provided that the resulting approximation is clearly documented. With these improvements, the platform could scale beyond the current local batch-processing environment while retaining data quality, traceability, and reproducibility.